DIPLOMA IN CYBERSECURITY / INFORMATION TECHNOLOGY

UNIT 14

Man-in-the-Middle (MITM) Attacks

Isolated Lab Demonstration | TLS and Certificate Validation

Complete Lecture & Student Handout

WARNING: Legal and ethical reminder

Every technique in this unit must run only inside the isolated lab supplied with these notes (all traffic stays on 127.0.0.1 — your own machine talking to itself). Intercepting someone else's traffic — on a shared Wi-Fi network, a college LAN, or anywhere you do not own and do not have written permission to test — is a criminal offence under the Computer Misuse and Cybercrimes Act, 2018 (Kenya) and equivalent laws elsewhere, and it is also a serious breach of other people's privacy.

Lesson Overview and Timing Plan

Learning outcomes

By the end of this session, learners should be able to:

1. Define a Man-in-the-Middle (MITM) attack and explain the position an attacker must reach to carry one out.
2. Explain how ARP, DNS and Wi-Fi association normally work, and how each can be abused to put an attacker in the middle.
3. Describe the main MITM techniques: ARP spoofing, DNS spoofing, rogue access points (evil twin), DHCP spoofing, SSL stripping and TLS-terminating interception.
4. Carry out a guided MITM demonstration in an isolated lab, showing plaintext credential interception and a TLS-terminating proxy.
5. Explain the TLS handshake and what certificate validation actually checks (chain of trust, expiry, hostname, revocation).
6. Explain why correct certificate validation defeats a TLS-terminating MITM attack, and demonstrate this directly.
7. Identify common certificate-validation failures (disabling verification, ignoring warnings, accepting self-signed certificates) and their consequences.
8. List detection methods and defences against MITM attacks at the network, transport and application layers.
9. Apply legal and ethical rules before any interception activity, in the lab and in professional practice.

Prerequisites and lab requirements

- **Prior knowledge:** TCP/IP and the OSI model, IP addressing, HTTP/HTTPS basics (Unit 6), cookies and sessions (Units 6 and 9).
- **Software:** Python 3.9+ with Flask (`pip install flask`), OpenSSL (for generating the lab certificates — already provided), curl, and a text editor.
- **Lab files (supplied with these notes, listed in Appendix A):** `bank_http_server.py`, `mitm_http_proxy.py`, `bank_https_server.py`, `tls_mitm_proxy.py`, `secure_client.py`, `insecure_client.py`, `run_lab_demo.sh`, and a `certs/` folder containing two self-signed certificate pairs.
- **Network:** everything in this lab runs on **127.0.0.1 (localhost)** — the learner's own machine talking to itself on different ports. No real network, Wi-Fi hardware, or second machine is required, and nothing ever leaves the computer.

Quick setup and self-test (run before class)

```
python -m venv venv
source venv/bin/activate          # Windows: venv\Scripts\activate
pip install flask
bash run_lab_demo.sh              # runs the whole demonstration end to end
```

1. Introduction to Man-in-the-Middle Attacks

1.1 What is a MITM attack?

A **Man-in-the-Middle (MITM) attack** happens when an attacker secretly positions themselves **between two parties who believe they are communicating directly with each other**, and relays, reads, or alters the traffic passing between them. Neither party notices anything unusual: the victim's browser still shows a web page, the server still receives what looks like a normal request.

Analogy: you write a sealed letter to a friend and hand it to a postal worker to deliver. If a dishonest worker secretly opens every letter, reads it, perhaps changes a sentence, reseals it and delivers it, neither you nor your friend would know, unless the seal, the paper, or something about the letter itself proved it had been tampered with. **TLS and certificate validation are that proof.**

> **KEY POINT: The core idea**
> MITM is not one specific technique; it is a **position**. An attacker who can get traffic routed, or trusted, through a point they control can then apply any number of techniques — reading, logging, blocking, or rewriting data in transit.

1.2 Two separate problems, often confused

Problem	Question it answers	What solves it
Getting into the path	How does the attacker's traffic-relaying position get established in the first place?	ARP spoofing, DNS spoofing, rogue Wi-Fi, a compromised router, a malicious proxy (Section 3)
Reading/altering the traffic once in the path	Once the attacker sees the traffic, can they actually understand or change it?	Encryption and authentication: TLS with correct certificate validation (Section 5)

This unit teaches both halves, and the lab demonstrates the second half directly: **being in the path is necessary but not sufficient**. An attacker who intercepts strongly encrypted, correctly validated traffic sees only ciphertext they cannot read or usefully alter.

1.3 Where MITM fits with earlier units

- **Unit 4 (Network scanning)** showed how an attacker maps a network. A MITM position is often set up on that same local network.
- **Unit 6 (Web architecture)** introduced HTTPS, TLS and cookies. This unit explains exactly what TLS is protecting against.
- **Unit 9 (CSRF)** showed an attack delivered through the victim's own browser. MITM is different: the attacker does not need the victim to visit a malicious page at all; the attacker sits on the **network path** itself.

1.4 What MITM can and cannot do

MITM CAN (when successfully positioned)	MITM CANNOT (against correctly implemented TLS)
Read and log plaintext traffic (HTTP, unencrypted email, Telnet, many IoT protocols)	Read the content of correctly validated, strongly encrypted TLS traffic
Inject or alter content in plaintext connections (ads, malware, modified pages)	Forge a certificate for a domain it does not control, signed by a CA the victim's client trusts
Attempt to downgrade a connection from HTTPS to HTTP (SSL stripping)	Defeat HSTS once a browser has already recorded that a site must always use HTTPS
Capture session cookies and credentials sent without encryption	Silently pass a TLS-terminating proxy past a client that properly checks the certificate chain and hostname

2. Foundations: What Makes MITM Possible

MITM attacks exploit **trust that protocols place in unauthenticated information**. This section revisits three mechanisms from a single angle: “what does this protocol trust, and what happens if that trust is abused?”

2.1 ARP: trusting whoever answers first

Recall from Unit 4 that on a local network, devices use the **Address Resolution Protocol (ARP)** to map IP addresses to MAC addresses. ARP has **no authentication**: any device on the network segment can answer an ARP request, or even send an unsolicited ARP reply, and other devices will update their ARP tables to match, because ARP was designed in an era when the local network was assumed to be trustworthy.

Figure 2.1: ARP spoofing in one picture

```
Normal:   Alice asks "who has 192.168.1.1?"  ->  Router replies "192.168.1.1 is at
AA:AA:AA:AA:AA:AA"
          Alice sends traffic for 192.168.1.1 to MAC AA:AA:AA:AA:AA:AA  (the real router)

Spoofed:  Attacker sends unsolicited reply "192.168.1.1 is at EE:EE:EE:EE:EE:EE" (the
attacker's MAC)
          Alice believes it and updates her ARP table
          Alice now sends traffic for the router -- straight to the attacker
```

2.2 DNS: trusting whoever answers fastest

DNS translates names to IP addresses. A DNS **query** is typically sent over UDP with a 16-bit transaction ID and no authentication by default. If an attacker can see the query (for example, because they are already on the same LAN via ARP spoofing) or can simply guess or flood responses, they can return a **forged answer** before the real DNS server replies, redirecting the victim’s browser to an address the attacker controls.

Technique	How
Local DNS spoofing	Attacker on the same LAN answers DNS queries directly (often combined with ARP spoofing first)
DNS cache poisoning	Attacker tricks a recursive resolver into caching a forged record, affecting every user of that resolver
Rogue DHCP	Attacker’s DHCP server hands out itself as the DNS server to new devices joining the network (Section 3.4)

DNSSEC (DNS Security Extensions) adds cryptographic signatures to DNS records so resolvers can verify authenticity, but it is still not universally deployed or enforced by every client, which is why DNS spoofing remains a practical technique.

2.3 Wi-Fi association: trusting a familiar name

Devices commonly reconnect automatically to Wi-Fi networks they have used before, identified only by the network name (**SSID**). Nothing in that process cryptographically proves that an access point broadcasting “CampusWiFi” is the institution’s genuine access point rather than an attacker’s laptop broadcasting the same name with a stronger signal nearby. This is the basis of the **evil twin / rogue access point** attack (Section 3.3).

2.4 TLS: built specifically to prevent MITM

Unlike ARP, DNS (without DNSSEC) and open Wi-Fi association, **TLS was explicitly designed to resist a network-level attacker**. Revisiting Unit 6's introduction, TLS provides three guarantees: **confidentiality** (encryption), **integrity** (tamper-evidence) and **authentication** (proof of server identity via a certificate). The third guarantee is what defeats MITM, and it only works if the client actually **validates** the certificate.

2.4.1 The TLS handshake, focused on identity

Figure 2.2: Simplified TLS 1.3 handshake

```
Client                                                Server
  |-- ClientHello (supported TLS versions, ciphers) ->
  |<- ServerHello + CERTIFICATE + key exchange data --|
  |   (certificate contains: subject name, public key,|
  |    issuer / CA, validity dates, digital signature) |
  |-- [CLIENT CHECKS THE CERTIFICATE HERE]             |
  |-- key exchange, Finished ------------------------->
  |<- Finished ----------------------------------------|
  |== now an encrypted channel, IF validation passed ==|
```

2.4.2 What certificate validation actually checks

Check	Question answered	What happens if it is skipped
Chain of trust	Was this certificate signed by a Certificate Authority (CA) that is in my trusted root store, directly or through intermediates?	Any attacker can present a self-signed or privately-issued certificate and be accepted
Validity period	Is today's date between the certificate's "not before" and "not after" dates?	Expired or not-yet-valid certificates (including attacker-crafted ones) are accepted
Hostname match	Does the certificate's subject / Subject Alternative Name list include the exact hostname I am connecting to?	A valid certificate for a *different*, attacker-owned site could be presented for this connection
Revocation status	Has the issuing CA revoked this certificate (for example after a key compromise), via OCSP or a CRL?	A stolen or compromised certificate keeps being accepted even after the owner has reported it
Signature validity	Does the cryptographic signature on the certificate actually verify against the issuer's public key?	A tampered or forged certificate structure could pass as genuine

> **WARNING: The one-sentence version**
>
> A certificate only proves identity if **something the client already trusts** vouches for it. A certificate an attacker generated themselves (a self-signed certificate, or one signed by a CA the client does not trust) proves nothing except that someone, somewhere, ran OpenSSL — which is exactly what the lab demonstrates in Section 5.

3. MITM Techniques

Each technique below answers the "getting into the path" half of the problem (Section 1.2). Several can be combined — for instance, ARP spoofing is routinely followed by DNS spoofing and then SSL stripping.

3.1 ARP spoofing / ARP poisoning

The attacker sends forged ARP replies to the victim (claiming to be the router) and, in a full attack, forged ARP replies to the router (claiming to be the victim), so traffic in **both directions** flows through the attacker, who typically forwards it onward so the connection keeps working while being observed. Classic tools: **Ettercap**, **Bettercap**, **arpspoof** (part of dsniff). This only works on the same local network segment (Unit 4, Section 2.5).

- **Requires:** attacker already connected to the same LAN or Wi-Fi.
- **Typically paired with:** packet sniffing (Wireshark), SSL stripping, or a TLS-terminating proxy like the one in this unit's lab.

3.2 DNS spoofing

Once an attacker can see or inject into a victim's DNS queries (often after ARP spoofing), they answer with a forged IP address, redirecting the victim to an attacker-controlled server that may look identical to the real site.

3.3 Rogue access point / "evil twin" Wi-Fi

The attacker sets up a wireless access point broadcasting the **same or a very similar SSID** as a legitimate network (a café, airport, hotel or campus Wi-Fi), often with no password or an open captive portal, to attract victims to connect directly through the attacker's own equipment — no ARP spoofing needed, because the attacker **is** the network.

Variant	Description
Open evil twin	Simple open network with an attractive, familiar-looking name
Karma / probe-response attack	Access point automatically responds to any network name a nearby device is probing for (devices often broadcast networks they remember), making it appear to be a network the victim has used before
Captive portal phishing	Victim is shown a fake login page (for the Wi-Fi itself, or imitating a known service) to harvest credentials directly

3.4 DHCP spoofing

A rogue DHCP server on the network races the real one to answer a new device's DHCP request, and if it wins, it can hand out itself as the **default gateway** and/or **DNS server**, putting it in the middle of all of that device's traffic without touching ARP at all.

3.5 SSL stripping (HTTPS downgrade)

Even when a site supports HTTPS, many users start by typing a bare domain name or clicking an old HTTP link. An attacker already in the path (via ARP/DNS spoofing) can intercept the initial **plain HTTP** request and

the server's redirect to HTTPS, and instead quietly proxy the connection: **HTTP to the victim, HTTPS to the real site**, rewriting links so the victim's browser never actually negotiates TLS. The browser's address bar shows `http://`, which attentive users might notice, but many do not. **HSTS (HTTP Strict Transport Security)**, covered in Unit 6, is the specific defence: once a browser has seen a site's HSTS header (or the site is on the browser's HSTS preload list), the browser refuses to connect over plain HTTP at all, closing this gap.

3.6 TLS-terminating interception (what this unit's lab demonstrates)

A more capable attacker (or a legitimate but security-relevant product — see the note below) does not merely relay plaintext; they **terminate the victim's TLS connection themselves**, using a certificate they control, decrypt and read the traffic, then open a **second, separate TLS connection** onward to the real server, re-encrypting. This is exactly the architecture of commercial "SSL inspection" proxies, corporate content filters, some antivirus products, and criminal MITM tooling alike — the technique is identical; **only authorisation and intent differ**.

3.7 Other MITM-adjacent techniques (brief)

Technique	One-line description
ARP spoofing + packet sniffing only (passive)	Attacker reads traffic without altering it; harder to detect than active modification
BGP hijacking	Large-scale, internet-routing-level MITM where an autonomous system announces routes it should not, redirecting traffic for entire IP ranges; outside the scope of this lab but important to know exists
Malicious/compromised Wi-Fi routers or firmware	A legitimate-looking router that has been compromised or shipped with built-in interception (see the Superfish case in Section 6)
Browser extension / local proxy malware	Malware already on the victim's device that intercepts traffic locally before it even reaches the network, sometimes by installing a rogue root certificate

4. Lab Demo 1: Intercepting Plaintext Traffic

This demo shows the simplest, oldest form of MITM: an attacker sitting in the path of **unencrypted** traffic needs no cleverness at all beyond being there. Everything runs on `127.0.0.1`; different **ports** stand in for different machines on a real network.

4.1 The scene

Component	File	Port	Role
The bank	`bank_http_server.py`	6000	A legacy site that still accepts logins over plain HTTP
The attacker's proxy	`mitm_http_proxy.py`	6001	Forwards traffic to the bank while logging everything it sees

In a real attack, step one (getting the victim's traffic to flow via port 6001-equivalent instead of straight to port 6000-equivalent) would be ARP spoofing, a rogue access point, or DNS spoofing (Section 3). In this lab we skip straight to **"the victim is already routed through the attacker"** so we can focus on what happens next.

4.2 The code, in brief

Core of mitm_http_proxy.py (Appendix A.2)

```
def pump(src, dst, tag):
    while True:
        data = src.recv(4096)
        if not data:
            break
        text = data.decode(errors="replace")
        m = re.search(r"username=([^&\s]+)&password=([^&\s]+)", text)
        if m:
            print(f"[MITM] *** CAPTURED CREDENTIALS *** username={m.group(1)}
password={m.group(2)}")
        dst.sendall(data)        # forward unchanged, so the victim notices nothing
```

The proxy is deliberately simple: it opens a connection to the real bank for every victim connection, and copies bytes in both directions, inspecting them as they pass. It never needs to crack any encryption, because **there is none to crack**.

4.3 Running it

1. Start the bank: `python bank_http_server.py` (listens on `127.0.0.1:6000`).
2. Start the attacker's proxy: `python mitm_http_proxy.py` (listens on `127.0.0.1:6001`, forwards to `6000`).
3. Act as the **victim**, submitting a login that is routed through the attacker: `curl -d "username=alice&password=alice123" http://127.0.0.1:6001/login`
4. Show the victim's own terminal: it receives a normal-looking "Logged in!" response — **nothing seems wrong**.
5. Switch to the attacker's proxy console and show what it logged.

Verified output, captured while preparing this unit

```
$ curl -s -d "username=alice&password=alice123" http://127.0.0.1:6001/login | grep -o "Logged
in!\|Invalid login"
Logged in!


--- attacker's proxy console ---
[MITM] listening on 127.0.0.1:6001, forwarding to 127.0.0.1:6000
[MITM] *** CAPTURED CREDENTIALS *** username=alice password=alice123
```

4.4 What just happened, and why it matters

- Alice's password travelled across the network in **plain, readable text**. Anyone able to see that traffic — not just a deliberate attacker's proxy, but any packet capture on a shared network, a compromised router, or a curious person on the same public Wi-Fi — could read it.
- The victim had **no way to notice** anything was wrong: no browser warning, no broken page, because nothing in plain HTTP checks who is actually on the other end of the connection.
- This is precisely why **HTTPS matters everywhere**, not only on "sensitive" pages: any page that later sets a cookie, submits a form, or loads a script should be served over TLS, because a plaintext page is a MITM target even if it looks harmless.

5. Lab Demo 2: TLS and Certificate Validation

This is the heart of the unit. We rebuild the **same** MITM idea against an HTTPS-protected version of the bank, and show two different clients behaving very differently: one that validates certificates correctly, and one that does not.

5.1 The scene

Component	File	Port	Certificate used
The real bank (HTTPS)	`bank_https_server.py`	6443	`certs/bank_cert.pem` — the bank's own genuine key pair
The attacker's TLS-terminating proxy	`tls_mitm_proxy.py`	6444	`certs/attacker_cert.pem` — a certificate the attacker generated themselves, also claiming to be "bank.local"
A properly configured client	`secure_client.py`	—	Trusts only `certs/bank_cert.pem` and checks the hostname (default, correct behaviour)
A misconfigured client	`insecure_client.py`	—	Disables certificate and hostname checking entirely (a common real-world mistake)

Both certificates claim the same identity, `CN=bank.local`, just as a real attacker running a TLS-intercepting proxy would have to create a certificate claiming to be the target site. The difference is **who is trusted to vouch for that claim**.

5.2 How the attacker's TLS proxy works

Core structure of tls_mitm_proxy.py (Appendix A.5) — this is also how commercial "SSL inspection" proxies work

```
# 1. Terminate the victim's TLS connection using the ATTACKER'S OWN certificate/key
server_ctx = ssl.SSLContext(ssl.PROTOCOL_TLS_SERVER)
server_ctx.load_cert_chain("certs/attacker_cert.pem", "certs/attacker_key.pem")
client_tls = server_ctx.wrap_socket(client_raw, server_side=True)

# 2. Open a SEPARATE, new TLS connection to the REAL bank
upstream_tls = upstream_ctx.wrap_socket(upstream_raw, server_hostname=TARGET_HOST)

# 3. Copy decrypted bytes both ways, logging anything that looks like credentials
#    (same pump() logic as Demo 1, just now operating on DECRYPTED data)
```

This two-connection design — one TLS session to the victim, a completely separate TLS session to the real server — is exactly what lets the proxy read and even modify traffic that is, on each individual hop, properly encrypted. The security of the **whole path** depends entirely on whether the victim's client was willing to accept the attacker's certificate in step 1.

5.3 Scenario A: a properly validating client meets the real bank directly

Verified output — baseline, no attacker in the path

```
$ python secure_client.py 6443

Connected and logged in successfully.
Server said: Logged in!
```

5.4 Scenario B: the SAME properly validating client, routed through the attacker's proxy

Verified output — the attack is blocked

```
$ python secure_client.py 6444

CONNECTION REFUSED by the client: certificate validation failed.
  -> [SSL: CERTIFICATE_VERIFY_FAILED] certificate verify failed: self-signed certificate
(_ssl.c:1000)
```

The client's trust store contains only the bank's genuine certificate. The attacker's certificate, however convincing it looks (same `CN=bank.local`, same key size, same format), was **not signed by anything the client trusts**, so the TLS library itself refuses to proceed. No application code had to add this check — `ssl.create_default_context()` validates by default, exactly as a browser does.

5.5 Scenario C: a misconfigured client, routed through the same attacker's proxy

Verified output — captured during preparation of this unit. Note: the proxy log shows TWO different clients in sequence: secure_client.py being rejected, then insecure_client.py succeeding

```
$ python insecure_client.py 6444

TLS handshake completed with NO certificate validation
(received a 806-byte certificate and accepted it blindly).
Server said: Logged in!


--- attacker's TLS proxy console ---
[TLS MITM] listening on 127.0.0.1:6444 (TLS), forwarding to 127.0.0.1:6443 (TLS)
[TLS MITM] victim's client REJECTED our certificate: [SSL: TLSV1_ALERT_UNKNOWN_CA] tlsv1 alert
unknown ca (_ssl.c:1000)
[TLS MITM] *** CAPTURED CREDENTIALS (decrypted!) *** username=alice password=alice123
```

The only difference between `secure_client.py` and `insecure_client.py` is two lines:

The entire vulnerability, in Appendix A.7

```
ctx.check_hostname = False      # <-- the mistake
ctx.verify_mode = ssl.CERT_NONE  # <-- the mistake
```

> **WARNING: This is not a contrived example**
>
> These exact two settings (or their equivalents — `verify=False` in Python's `requests` library, `-k`/`--insecure` in curl, "trust all certificates" helper classes, or a user clicking through a browser warning) appear **routinely** in real codebases, usually added temporarily to work around a certificate problem during development and then **never removed**. Penetration testers find this pattern constantly in mobile apps and internal tools.

5.6 Running the whole thing in one go

`run_lab_demo.sh` (Appendix A.8) runs Demo 1 and all three scenarios of Demo 2 in sequence, starting and stopping every server automatically, so the whole comparison can be shown live in under two minutes once rehearsed:

Verified end-to-end run

```
$ bash run_lab_demo.sh

### PART 1: Plaintext HTTP -- MITM reads credentials directly ###
... Logged in!
[MITM] *** CAPTURED CREDENTIALS *** username=alice password=alice123

### PART 2: HTTPS with PROPER certificate validation -- MITM is blocked ###
CONNECTION REFUSED by the client: certificate validation failed.
  -> [SSL: CERTIFICATE_VERIFY_FAILED] certificate verify failed: self-signed certificate

### PART 3: HTTPS with certificate validation DISABLED -- MITM succeeds ###
... Server said: Logged in!
[TLS MITM] *** CAPTURED CREDENTIALS (decrypted!) *** username=alice password=alice123

Demo complete.
```

5.7 The lesson in one paragraph

KEY POINT

Being positioned in the network path (Section 3) is only half an attack. Against plaintext traffic, that is already enough to steal credentials (Demo 1). Against TLS-protected traffic, the attacker also needs the victim's client to **accept a certificate it should not trust** — and a client that validates certificates correctly, the way browsers do by default, simply refuses to proceed. Every real-world TLS MITM success story (Section 6) comes down to defeating or bypassing that one check, not breaking the underlying cryptography.

CLASS ACTIVITY: Guided discussion during the demo

After Scenario B (blocked), question: "The attacker has the exact same position in the network as before. Why did this attack fail when the plaintext one succeeded?" Expected answer: TLS added **authentication of identity**, and the client actually checked it.

After Scenario C (succeeded), question: "What real-world situations make people, or software, disable certificate checking?" Expected answers: developers silencing an annoying warning, apps connecting to self-signed internal servers, users clicking "proceed anyway" on a browser warning, old or misconfigured IoT devices, and attackers deliberately exploiting all of the above.

6. Real-World Cases

REAL-LIFE SCENARIO: Lenovo Superfish (2015)

Lenovo shipped consumer laptops with pre-installed adware called Superfish that worked by installing its **own root certificate** into the operating system's trust store and intercepting HTTPS traffic to inject advertisements — exactly the architecture of `tls_mitm_proxy.py` in Section 5, running as "legitimate" software. Worse, Superfish used the **same private key on every affected laptop**, so once security researchers extracted it, anyone could use it to forge valid-looking certificates for any site and intercept traffic on every Superfish-equipped machine, including on open Wi-Fi networks. The case shows that installing a trusted root certificate is extremely powerful, and that doing it insecurely turns a privacy nuisance into a critical vulnerability.

REAL-LIFE SCENARIO: DigiNotar CA compromise (2011)

Attackers compromised the Dutch certificate authority DigiNotar and issued **fraudulent but technically valid certificates** for domains including google.com. Because DigiNotar was in major browsers' trusted root stores at the time, those forged certificates would pass exactly the chain-of-trust check described in Section 2.4.2, enabling undetected MITM interception of HTTPS traffic for affected users, reportedly used against users in Iran. The incident led browsers to revoke trust in DigiNotar entirely and accelerated work on **Certificate Transparency**, a system that publicly logs every certificate issued so misissuance can be spotted.

REAL-LIFE SCENARIO: Firesheep and open Wi-Fi session hijacking (2010)

The Firesheep browser extension made it trivial for anyone on an open Wi-Fi network to capture other users' unencrypted session cookies for popular sites that only used HTTPS for login but not for the rest of the session, then impersonate them. It was a plaintext MITM-style capture (conceptually identical to Demo 1, but of a session cookie rather than a password) that pushed the industry toward **HTTPS everywhere**, not just on login pages.

REAL-LIFE SCENARIO: Public Wi-Fi evil twins and captive-portal abuse (ongoing pattern)

Security researchers and penetration testers routinely demonstrate rogue access points at conferences, airports and hotels that mimic the venue's real network name and present a fake captive-portal login page, harvesting credentials directly (Section 3.3) before anyone's traffic is even analysed. Because this requires no software vulnerability at all, it remains one of the most reliable MITM techniques against the general public.

These cases differ in sophistication and in **who did it and why** — a crime, a CA breach, a researcher's proof-of-concept, or software marketed as legitimate — but every one of them comes back to the same two building blocks taught in this unit: getting into the traffic path, and then either exploiting a lack of encryption or defeating certificate validation.

7. Detection and Defence

7.1 Detecting MITM at the network layer

Technique	What it looks for
Static ARP entries / ARP monitoring (`arpwatch`, `arpalert`)	Unexpected changes in IP-to-MAC mappings, or one MAC address suddenly claiming to be the gateway
Switch port security / Dynamic ARP Inspection (DAI)	Managed switches can validate ARP against known DHCP leases and drop forged replies
DHCP snooping	Switches only allow DHCP replies from designated trusted ports, blocking rogue DHCP servers
Wireless intrusion detection (WIDS)	Flags access points broadcasting a known SSID from unauthorised hardware (evil twins)
Unusual latency or an extra network hop	A MITM proxy adds a small but sometimes detectable delay, and can show up in traceroute (Unit 4) as an unexpected hop

7.2 Detecting MITM at the application / TLS layer

- **Certificate warnings are not optional.** A browser or client refusing to connect, or showing "your connection is not private", is the system working correctly — the correct response is to stop, not to click through.
- **Certificate pinning** goes one step further than ordinary validation: the application hard-codes ("pins") exactly which certificate or public key it expects for a given server, so even a certificate from a technically trusted CA is rejected if it is not the pinned one. This defeats a compromised or coerced CA (as in the DigiNotar case) but adds operational complexity, since the pinned certificate must be updated before it expires.
- **Certificate Transparency (CT) logs** let anyone, including automated monitoring services, check whether a certificate has been issued for their domain that they did not request — an early-warning system against CA misissuance or compromise.
- **HSTS** (Unit 6) stops SSL-stripping downgrade attacks once a browser has seen the header (or the site is preloaded), because the browser will not attempt a plain-HTTP connection at all.

7.3 Defensive measures for individuals

- Avoid sensitive activity (banking, logins) on open or unfamiliar Wi-Fi; prefer a trusted personal hotspot or a VPN when it cannot be avoided.
- Never click through a certificate warning to a site handling sensitive data.
- Keep operating systems and browsers updated, since root certificate stores and revocation handling improve over time.
- Check for HTTPS (and, habitually, the correct domain name) before entering credentials anywhere.

7.4 Defensive measures for organisations

- **Enforce HTTPS everywhere** with HSTS, and redirect all HTTP to HTTPS server-side.

- **Never disable certificate validation** in production code, scripts, or mobile apps; treat any occurrence found in code review as a serious finding.
- **Harden the local network:** Dynamic ARP Inspection, DHCP snooping, 802.1X port-based authentication, and segmenting guest Wi-Fi away from internal systems.
- **Use WPA2-Enterprise/WPA3 with proper authentication** rather than a shared Wi-Fi password, which makes rogue-AP style attacks harder to disguise as “the real network”.
- **Deploy monitoring** (Unit 4’s SIEM/IDS content applies directly) to catch ARP anomalies, rogue DHCP servers and unexpected certificates.
- **Train staff and users** to treat certificate warnings as a stop sign, not an inconvenience to bypass.
- Where TLS inspection is genuinely required (for example corporate content filtering), deploy it **transparently and with informed consent**, document it, and keep the inspecting CA’s private key tightly controlled — the Superfish case is the cautionary tale for what happens when this goes wrong.

8. Activities, Discussion and Assessment

8.1 Hands-on lab: Run, Break, and Explain

Duration: about 60 minutes, using the supplied files (Appendix A), entirely on `127.0.0.1`.

1. Run Demo 1 manually (Section 4.3) and capture a screenshot of the attacker's proxy console showing a captured password.
2. Run Demo 2, Scenario A, B and C manually (Sections 5.3–5.5), capturing the output of each.
3. Open `certs/bank_cert.pem` and `certs/attacker_cert.pem` with `openssl x509 -in <file> -noout -text` and compare their **Subject**, **Issuer** and **Serial Number** fields. Explain in your own words why they look so similar yet behave so differently for `secure_client.py`.
4. Modify `secure_client.py` to use `ssl.create_default_context()` with **no** `cafile` argument (the system's normal trusted root store) instead of trusting `bank_cert.pem` directly, then try Scenario A again. Explain the result (the bank's certificate is self-signed and not in any public root store, so even this "normal" client now rejects it — discuss why real sites must use a CA-issued certificate, not a self-signed one, for exactly this reason).
5. Write a short report: what position would an attacker need on a **real** network to reach the point this lab starts from (Section 3), what the plaintext demo proved, what the TLS demo proved, and one specific recommendation each for (a) an end user and (b) a developer.

8.2 Discussion questions

1. Explain, in your own words, why MITM is described as a "position" rather than a single technique.
2. Why does ARP spoofing only work on the same local network segment, while DNS spoofing or a compromised CA can have a much wider impact?
3. A friend says, "My Wi-Fi has a password, so nobody can intercept my traffic." Evaluate this claim using what you learned about evil twins and about encryption versus network access.
4. Explain why the Superfish case was worse specifically because every affected laptop shared the same private key. What should Lenovo have done differently if local TLS inspection was genuinely needed?
5. Why does HSTS only protect a site after the first (successful, uncompromised) visit, and what closes that initial gap?
6. Using Section 2.4.2's table, identify which specific certificate check would catch each of the following: (a) an attacker presents a certificate for "yourbank.com" that expired last year; (b) an attacker presents a valid, current, CA-signed certificate — but one issued for "attacker-site.com", not your bank; (c) an attacker presents a certificate that is correctly formatted but entirely self-generated.
7. Why is "click through the warning" almost always the wrong response to a certificate error, even if the website "looks normal"?

8.3 Quick quiz (multiple choice).

Q1. A Man-in-the-Middle attack is best described as:

A. A type of malware that deletes files
B. An attacker secretly relaying/observing traffic between two parties who believe they are talking directly
C. A denial-of-service flood
D. A weak password attack

Q2. Which protocol has no built-in authentication and is commonly abused to redirect local traffic through an attacker?

A. TLS
B. ARP
C. DNSSEC
D. HSTS

Q3. An "evil twin" attack refers to:

A. Two attackers working together
B. A rogue access point broadcasting the same or a similar name as a legitimate Wi-Fi network
C. A duplicated TLS certificate
D. A second DNS server

Q4. SSL stripping specifically targets:

A. Encrypted database connections
B. The moment a connection would otherwise be upgraded from HTTP to HTTPS
C. ARP tables
D. DNS caches

Q5. In the lab, why did secure_client.py reject the connection through tls_mitm_proxy.py?

A. The proxy was offline
B. The attacker's certificate was not signed by anything the client trusted
C. The password was wrong
D. The port number was invalid

Q6. Which single setting in insecure_client.py is primarily responsible for the attack succeeding?

A. A weak password
B. ctx.verify_mode = ssl.CERT_NONE (and check_hostname = False)
C. Using port 6444 instead of 6443
D. Running as root

Q7. What does a TLS-terminating MITM proxy actually do?

A. It blocks all encrypted traffic
B. It ends the victim's TLS session with its own certificate, then opens a separate TLS session to the real server
C. It only works on plaintext traffic
D. It modifies DNS records only

Q8. What made the DigiNotar incident especially serious?

A. It was a phishing email campaign
B. A trusted Certificate Authority was compromised and issued fraudulent, technically valid certificates
C. It was a weak Wi-Fi password
D. It only affected one single user

Q9. Why was Lenovo's Superfish especially dangerous compared to a typical adware MITM proxy?

A. It had no certificate at all

B. The same private key was shared across every affected laptop, letting anyone forge valid-looking certificates

C. It only worked on Linux

D. It used no encryption whatsoever

Q10. HSTS primarily defends against:

A. ARP spoofing

B. SSL-stripping downgrade attacks

C. DHCP spoofing

D. CA compromise

Q11. Certificate pinning adds protection against:

A. Expired certificates only

B. A technically trusted but unexpected/wrong certificate being accepted, even from a legitimate CA

C. Weak passwords

D. ARP spoofing

Q12. Dynamic ARP Inspection (DAI) on a managed switch is a defence against:

A. SQL injection

B. ARP spoofing

C. SSRF

D. Cross-site scripting

Q13. Why does MITM remain dangerous even though TLS has existed for decades?

A. TLS is no longer used

B. Many systems still use plaintext protocols, and certificate validation is sometimes disabled or misconfigured

C. Attackers can break AES encryption directly

D. Browsers no longer check certificates

Q14. Which of these is NOT one of the standard certificate validation checks?

A. Chain of trust to a trusted CA

B. Hostname match

C. Validity period

D. The certificate file's size in kilobytes

Q15. The best response to an unexpected browser certificate warning on a banking site is to:

A. Click "proceed anyway" if the page looks normal

B. Stop, do not proceed, and investigate or use a trusted network/device instead

C. Disable your antivirus and retry

D. Switch to HTTP instead

8.4 Short-answer and applied questions

1. Define MITM and explain the two separate problems an attacker must solve to carry one out successfully. (4 marks)
2. Explain how ARP spoofing works and why it is limited to the local network segment. (5 marks)
3. Describe, step by step, what a TLS-terminating MITM proxy does, using the two-connection design from Section 5.2. (6 marks)
4. List the five checks a client performs during certificate validation (Section 2.4.2) and give one consequence of skipping each. (10 marks)

5. Explain why disabling certificate validation (as in insecure_client.py) is a realistic mistake in production software, not just a lab contrivance. (4 marks)
6. Compare the Superfish and DigiNotar cases: what is different about how trust was abused in each? (5 marks)
7. Design a short checklist (five items) that a code reviewer could use to catch disabled certificate validation before it reaches production. (5 marks)
8. Explain why HSTS alone would not have stopped the attack demonstrated in Demo 2 of this unit's lab. (3 marks)

9. Summary, Glossary and Quick Reference

9.1 Key takeaways

- **MITM is a position**, not a single technique: an attacker sits between two parties who believe they are talking directly, and can observe or alter traffic.
- Getting into that position exploits **unauthenticated trust** in protocols: ARP (trusts whoever answers), DNS without DNSSEC (trusts whoever answers first), and open Wi-Fi association (trusts a familiar-looking name).
- Once positioned, **plaintext traffic is trivially readable** (Demo 1) — this is why HTTPS everywhere, not just on login pages, matters.
- **TLS was built specifically to resist MITM**, through encryption, integrity protection and, crucially, **certificate-based authentication of the server's identity**.
- Certificate validation checks the **chain of trust, validity period, hostname match, revocation status and signature** — and this unit's lab proved directly that skipping these checks (as `insecure_client.py` does) is the entire difference between an attack failing and succeeding.
- Real incidents (Superfish, DigiNotar, Firesheep, evil twins) each trace back to the same two building blocks taught here.
- Defence is layered: network-level controls (DAI, DHCP snooping, WIDS) stop attackers getting into position; transport-level controls (HSTS, correct certificate validation, pinning, Certificate Transparency) stop them from doing anything useful even if they do.
- Practise only in an **isolated lab** (as supplied) or with **explicit written authorisation**; intercepting real traffic without consent is both illegal and a serious breach of trust.

9.2 Glossary

Term	Definition
ARP spoofing / poisoning	Sending forged ARP replies to redirect local traffic through an attacker
Certificate Authority (CA)	An organisation trusted to verify identity and issue digital certificates
Certificate pinning	Hard-coding the expected certificate or public key for a server, rejecting any other even if otherwise trusted
Certificate Transparency (CT)	Public logs of issued certificates, allowing detection of unauthorised or fraudulent issuance
Chain of trust	The sequence of signatures linking a certificate back to a trusted root CA
DHCP spoofing	Running a rogue DHCP server to redirect victims' gateway/DNS settings
DNS spoofing	Returning forged DNS responses to redirect a victim to an attacker-controlled address
DNSSEC	Cryptographic extensions to DNS that allow signature verification of records
Evil twin	A rogue Wi-Fi access point impersonating a legitimate network's name
HSTS	HTTP Strict Transport Security; forces a browser to only use HTTPS for a site after the first visit
MITM	Man-in-the-Middle; an attacker secretly positioned between two communicating parties

Term	Definition
Root store	The set of Certificate Authorities an operating system or browser trusts by default
SSL stripping	Intercepting the HTTP-to-HTTPS upgrade so the victim stays on an unencrypted connection
TLS handshake	The process by which a client and server agree on encryption and the server proves its identity
TLS-terminating proxy	A MITM technique where the attacker ends the victim's TLS session and opens a separate one to the real server

9.3 One-page cheat sheet

```
TWO HALVES OF A MITM ATTACK
  1. GET INTO THE PATH: ARP spoofing | DNS spoofing | evil twin Wi-Fi | DHCP spoofing
  2. EXPLOIT WHAT YOU SEE: plaintext = read directly | TLS = must defeat cert validation

CERTIFICATE VALIDATION CHECKLIST (what a client must verify)
  [ ] Chain of trust to a trusted root CA
  [ ] Validity period (not expired, not yet-valid)
  [ ] Hostname matches the certificate's subject / SAN
  [ ] Not revoked (OCSP / CRL)
  [ ] Signature cryptographically verifies

NEVER IN PRODUCTION CODE
  ssl.CERT_NONE | check_hostname = False | requests verify=False | curl -k/--insecure

DEFENCES
  Network:  Dynamic ARP Inspection, DHCP snooping, WIDS, 802.1X, segmentation
  Transport: HTTPS everywhere, HSTS, certificate pinning, Certificate Transparency
  People:   never click through a certificate warning; avoid sensitive use of open Wi-Fi
```

9.4 Self-study and further reading

- **OWASP Cheat Sheet Series**: "Transport Layer Security", "Pinning".
- **MDN Web Docs**: "Transport Layer Security (TLS)", "Strict-Transport-Security".
- **CA/Browser Forum** Baseline Requirements (current certificate lifetime and issuance rules — check for the latest version).
- **Certificate Transparency** (certificate.transparency.dev) for background on CT logs.
- **Tools to explore safely, in a lab only:** Wireshark (packet analysis), Ettercap/Bettercap (ARP spoofing, for authorised lab use), `arpwatch` (ARP monitoring).
- **Historical reading:** write-ups of the Superfish and DigiNotar incidents are widely documented by security researchers and worth reading in full for the operational detail.
- Certification pathways that build on this unit: CompTIA Security+ and PenTest+, CEH, and vendor-neutral network security courses.

Appendix A: Full Lab Code Listings

These files were written and verified while preparing this unit (all scenarios produced the output quoted in Sections 4 and 5). Copy each listing into a file of the same name in one folder, with a `certs/` subfolder for A.6.

A.1 bank_http_server.py

A.2 mitm_http_proxy.py

A.3 bank_https_server.py

A.4 tls_mitm_proxy.py

A.5 secure_client.py

A.6 Generating the lab certificates (certs/)

Inspect either certificate at any time with: `openssl x509 -in certs/bank_cert.pem -noout -text`

A.7 insecure_client.py

A.8 run_lab_demo.sh

ALL THE SOURCE CODE FILES WILL BE IN MY GITHUB REPOSITORY UNDER UNIT 14 DIRECTORY

Verified result when run: output matches Section 5.6 exactly (captured during preparation of this unit).